

time-series

Rishi Shanthan Bhagavatham

2024-10-08

R Commands

Exhibit 1.1 on page 2.

```
library(TSA)
```

```
##
```

```
## Attaching package: 'TSA'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      acf, arima
```

```
## The following object is masked from 'package:utils':
```

```
##
```

```
##      tar
```

```
quartz(width=4.875,height=2.5,points=8)
```

```
data(larain);
```

```
plot(larain,ylab='Inches',xlab='Year',type='o')
```

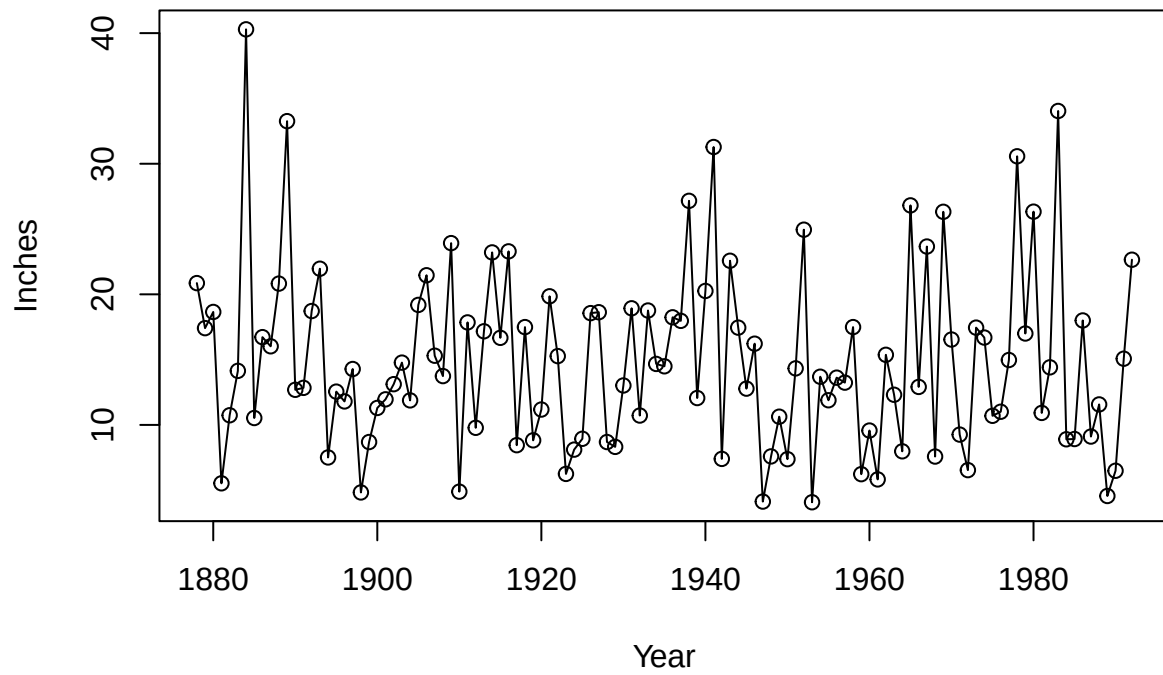


Exhibit 1.2 on page 2.

```
quartz(width=3,height=3,pointsize=8)
```

```
plot(y=larain,x=zlag(larain),ylab='Inches', xlab='Previous Year Inches')
```

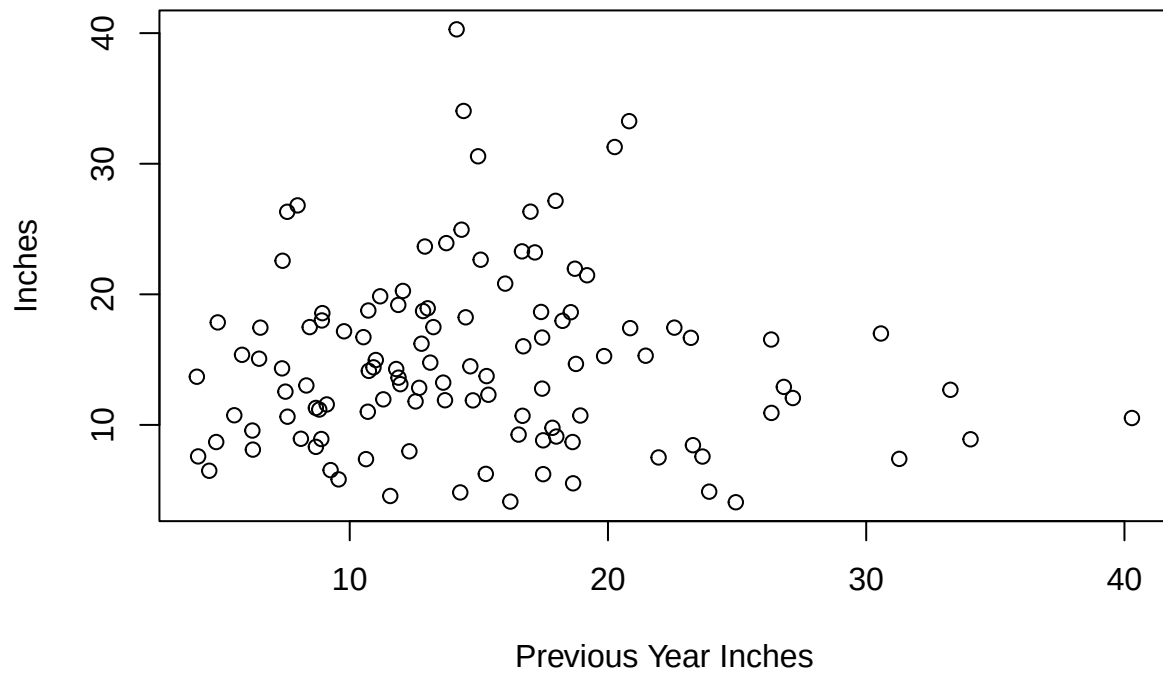
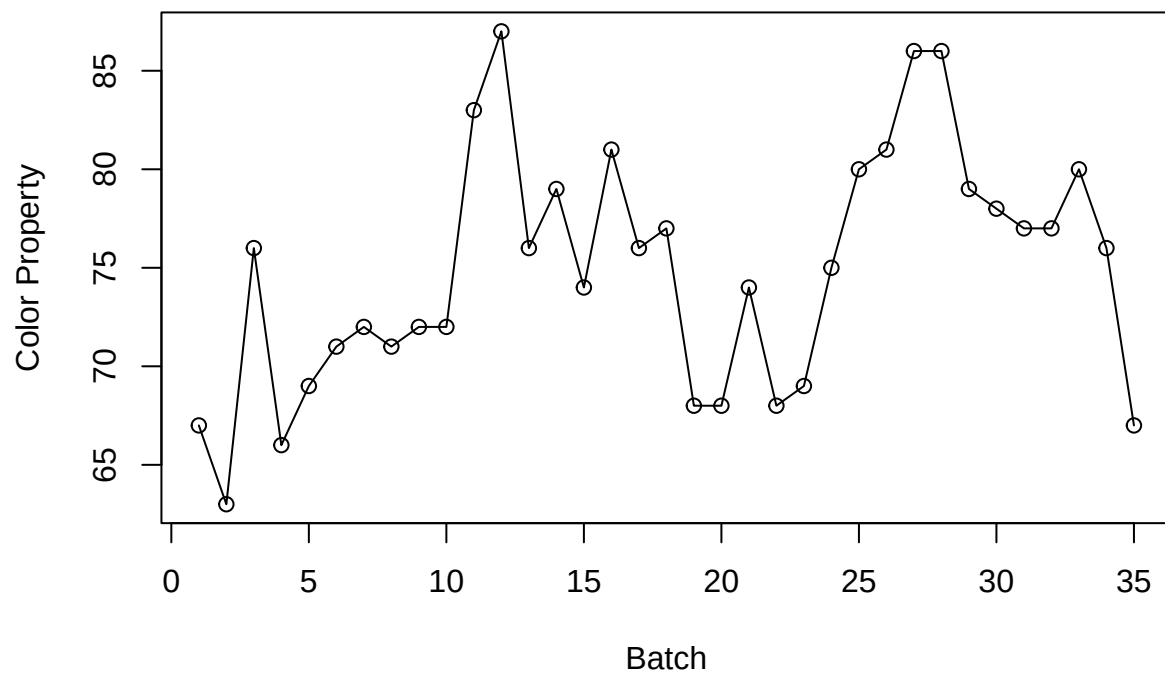
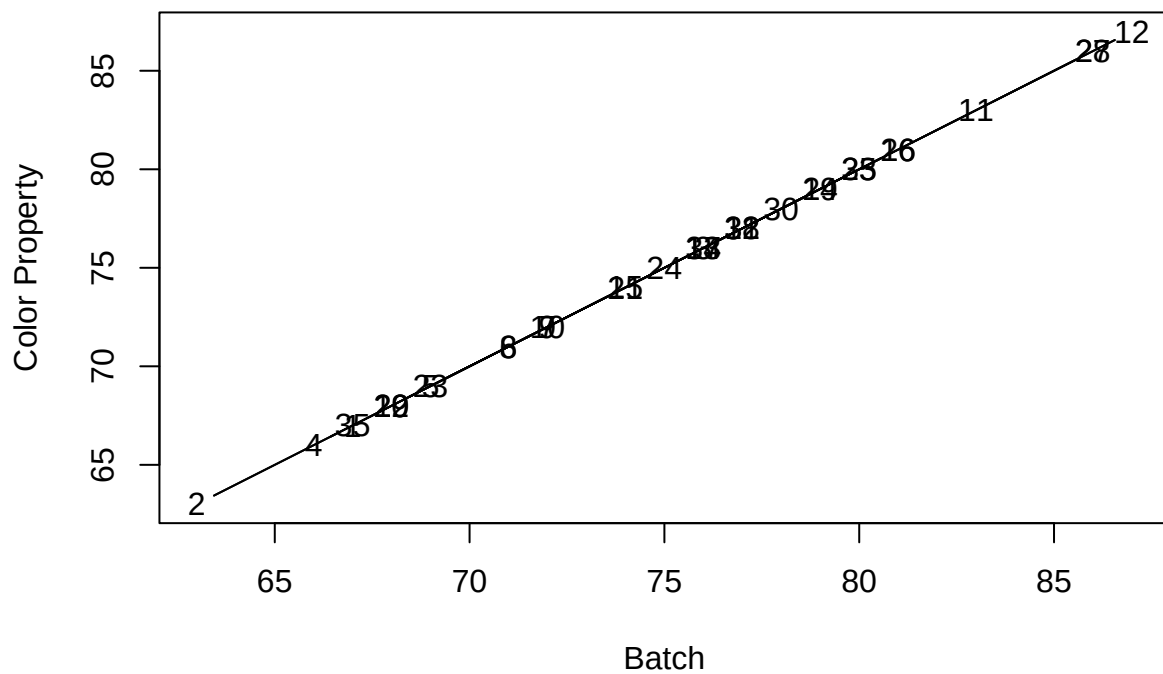


Exhibit 1.3 on page 3.

```
data(color)
plot(color,ylab='Color Property',xlab='Batch',type='o')
```



```
plot(color, color, ylab='Color Property',  
xlab='Batch',type='o')
```



```
as.vector(color)
```

```
## [1] 67 63 76 66 69 71 72 71 72 72 83 87 76 79 74 81 76 77 68 68 74 68 69 75 80
## [26] 81 86 86 79 78 77 77 80 76 67
```

```
plot(as.vector(color), color, ylab='Color Property', xlab='Batch',type='o')
```

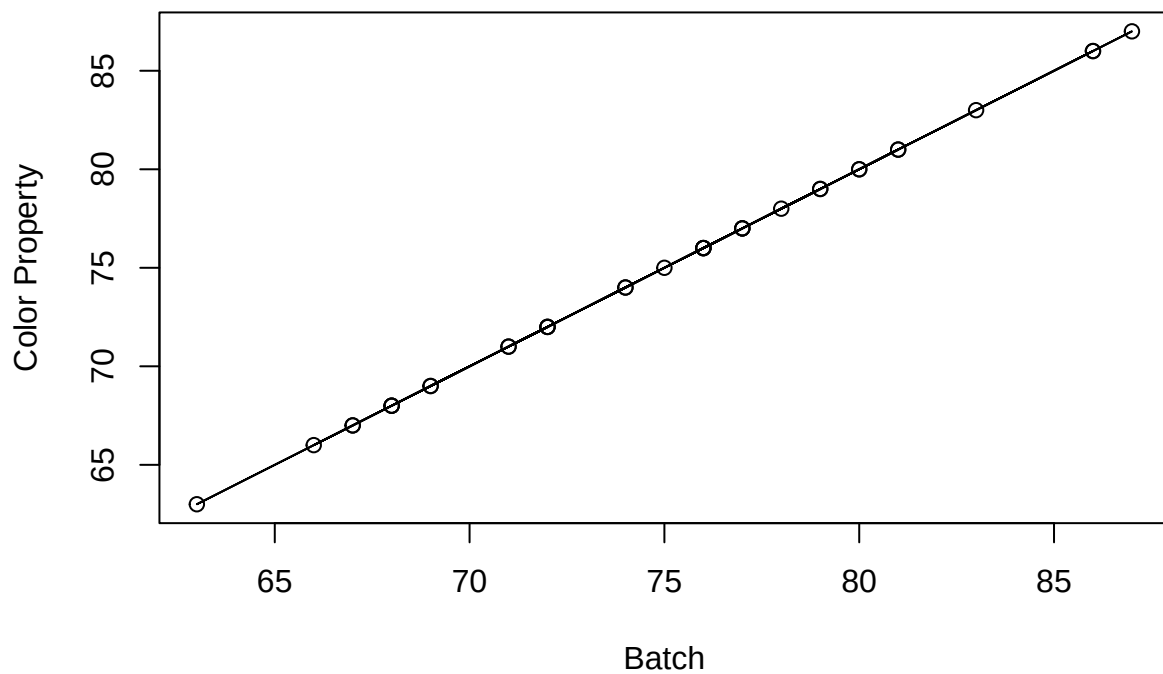


Exhibit 1.4 on page 4.

```
plot(y=color,x=zlag(color),ylab='Color Property', xlab='Previous Batch Color Property')
```

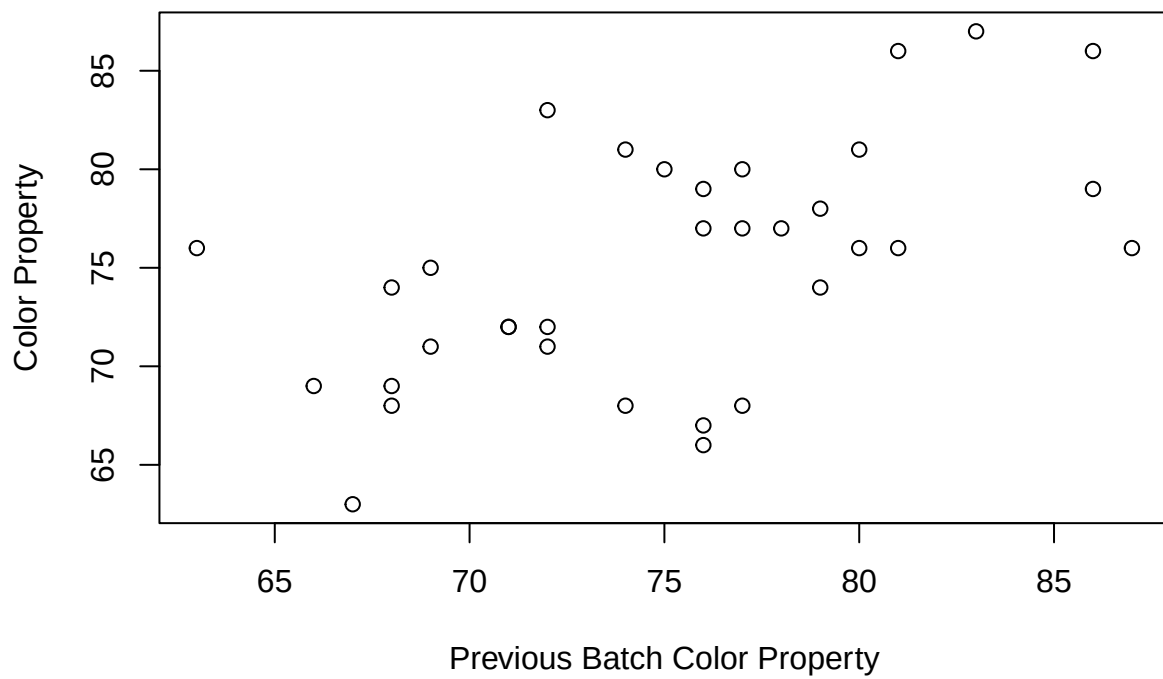
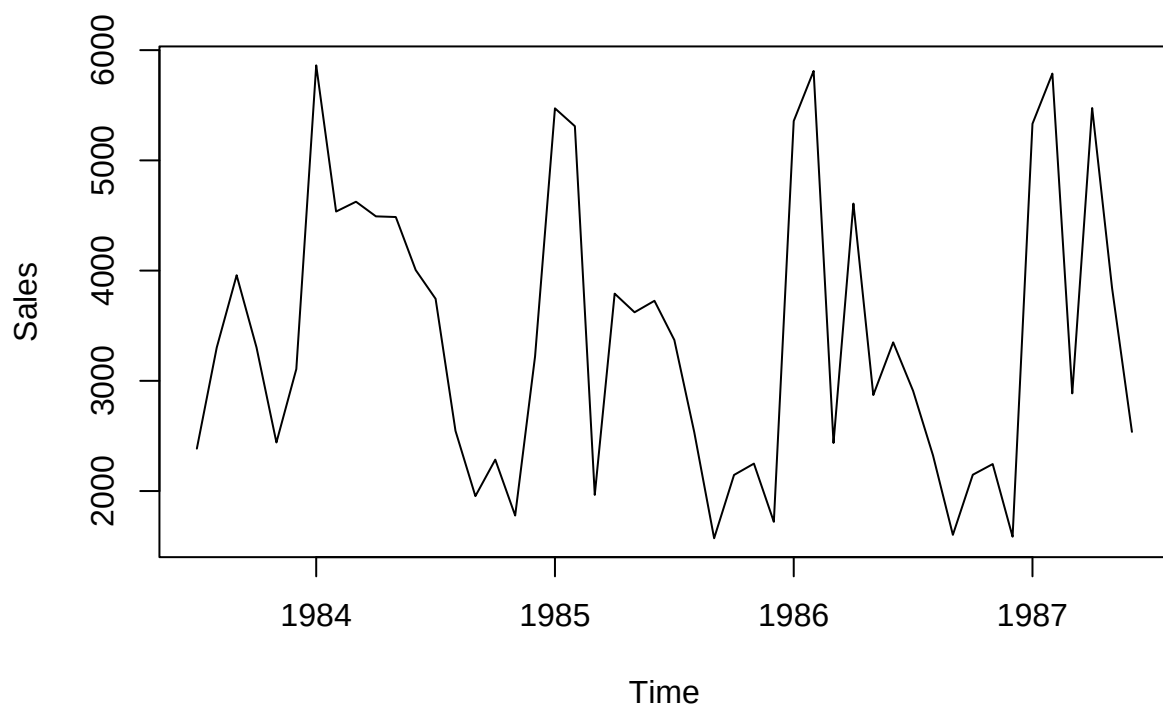
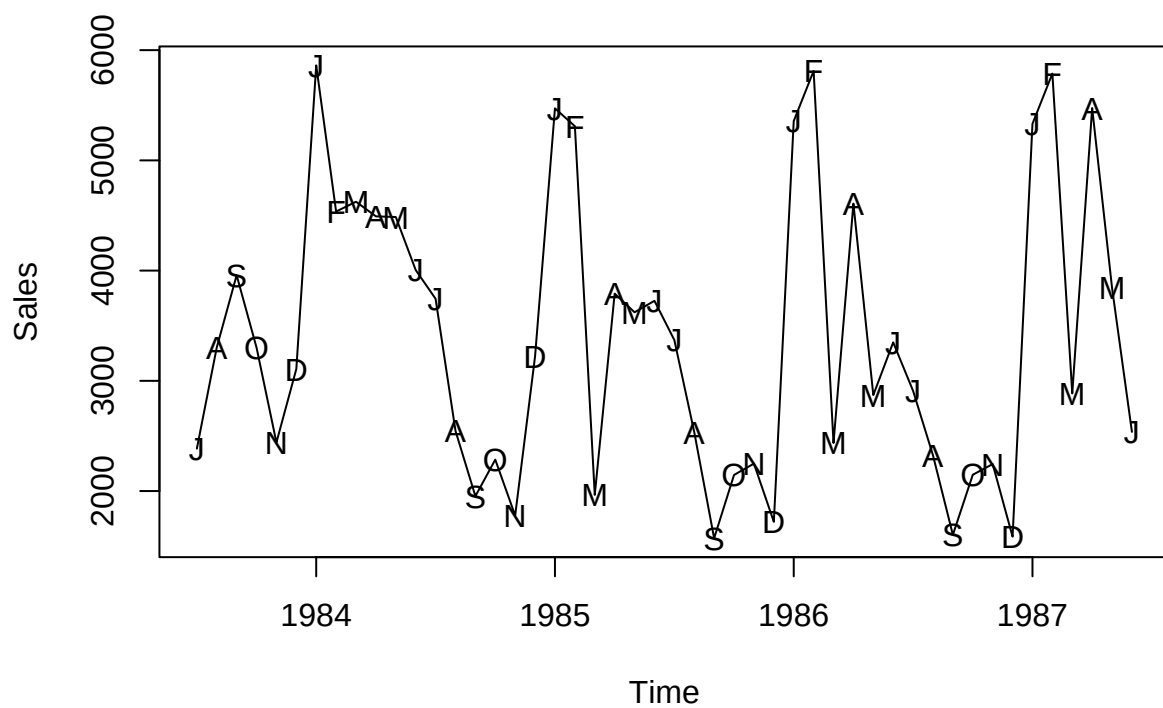


Exhibit 1.9 on page 7.

```
data(oilfilters)
plot(oilfilters,type='l',ylab='Sales')
```

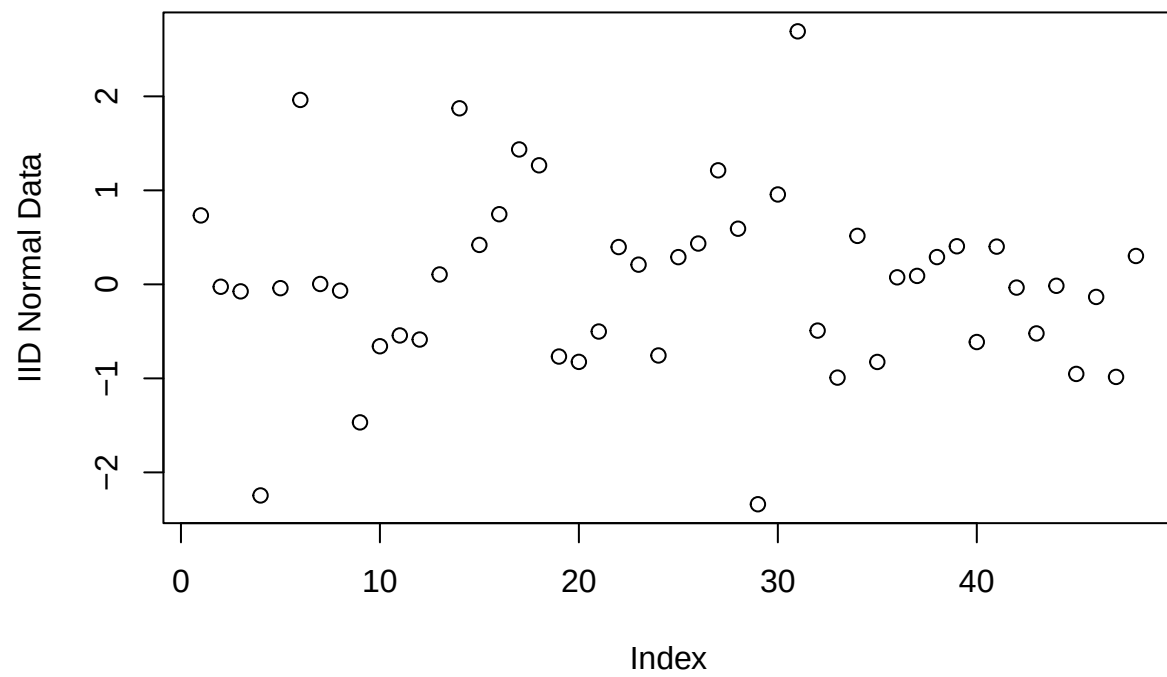


```
Month=c('J','A','S','O','N','D','J','F','M','A','M','J')
plot(oilfilters,type='l',ylab='Sales')
points(y=oilfilters,x=time(oilfilters),pch=as.vector(season(oilfilters)))
```



```
y=rnorm(48)
```

```
plot(y, type='p', ylab='IID Normal Data')
```



```
plot(y, type='o', ylab='IID Normal Data')
```

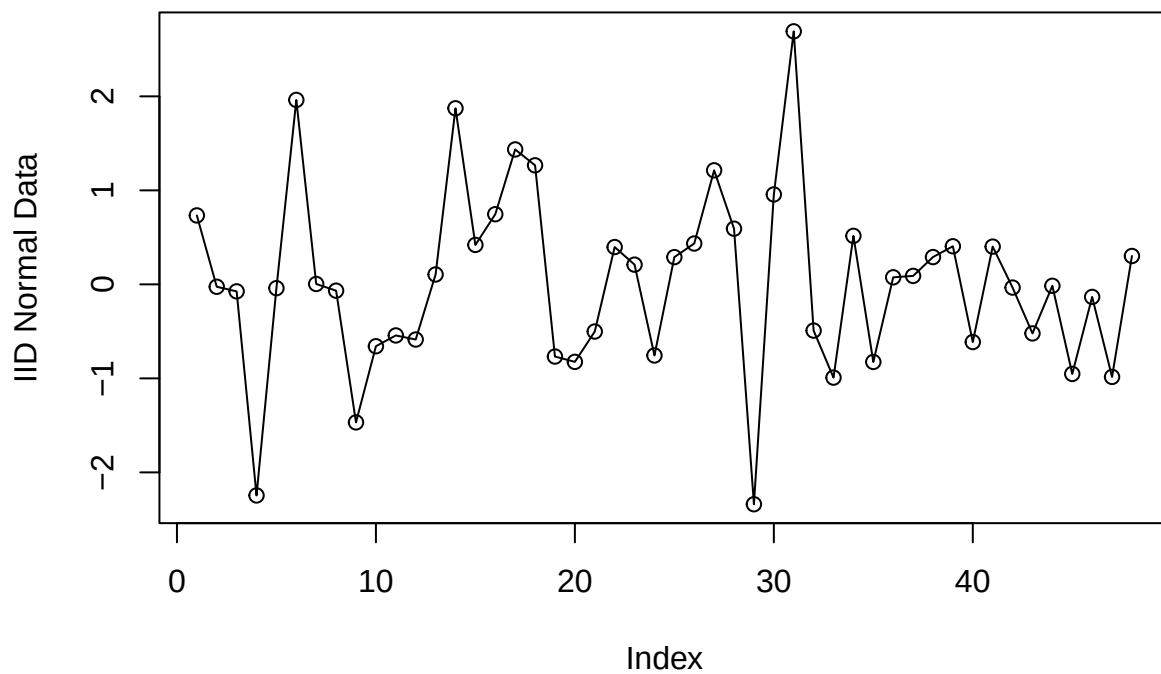


Exhibit 2.1 on page 14.

```
n=60
set.seed(12345)
sim.random.walk=ts(cumsum(rnorm(n)),freq=1,start=1)
plot(sim.random.walk,type='o',ylab='Another Random Walk')
```

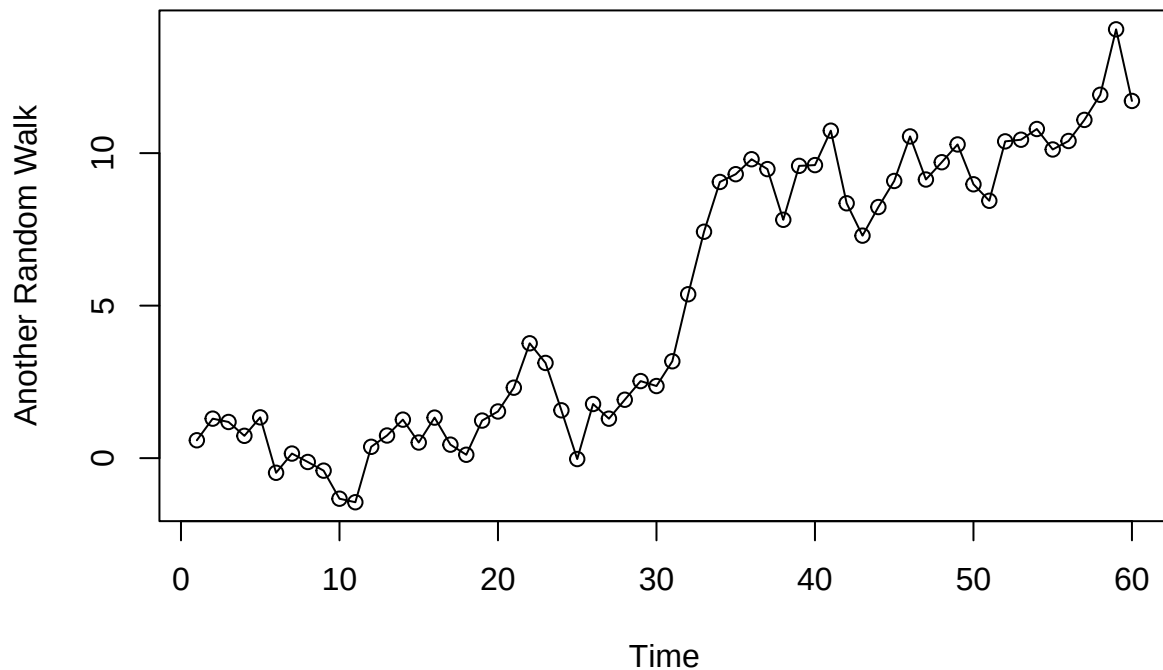


Exhibit 3.1 on page 31.

```
data(rwalk)
model1=lm(rwalk~time(rwalk))
summary(model1)
```

```
##
## Call:
## lm(formula = rwalk ~ time(rwalk))
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-2.70045	-0.79782	0.06391	0.63064	2.22128

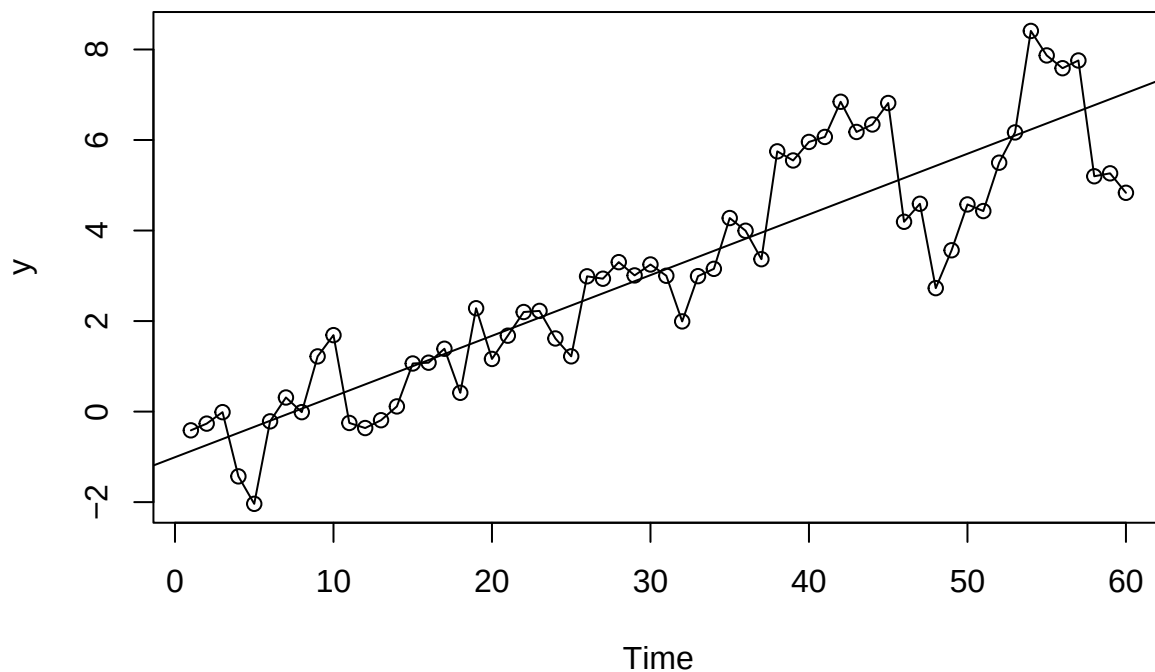
```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-1.007888	0.297245	-3.391	0.00126 **
time(rwalk)	0.134087	0.008475	15.822	< 2e-16 ***

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.137 on 58 degrees of freedom
## Multiple R-squared:  0.8119, Adjusted R-squared:  0.8086
## F-statistic: 250.3 on 1 and 58 DF, p-value: < 2.2e-16
```

Exhibit 3.2 on page 31.

```
plot(rwalk,type='o',ylab='y')
abline(model1)
```



```
model1a=lm(rwalk~time(rwalk)+I(time(rwalk)^2))
summary(model1a)
```

```
##
## Call:
## lm(formula = rwalk ~ time(rwalk) + I(time(rwalk)^2))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.69623 -0.76802  0.00826  0.85337  2.34468
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.427291   0.4534893  -3.147  0.00262 **
## time(rwalk)    0.1746746  0.0343028   5.092 4.16e-06 ***
## I(time(rwalk)^2) -0.0006654  0.0005451  -1.221  0.22721
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 1.132 on 57 degrees of freedom
## Multiple R-squared:  0.8167, Adjusted R-squared:  0.8102
## F-statistic: 127 on 2 and 57 DF,  p-value: < 2.2e-16
```

Exhibit 3.3 on page 32.

```
data(tempdub)
month.=season(tempdub)
sex=factor(c('M','F','M','M','F'),levels=c('M','F'))
sex
```

```
## [1] M F M M F
## Levels: M F
```

```
table(sex)
```

```
## sex
## M F
## 3 2
```

```
model2=lm(tempdub~month.-1)
summary(model2)
```

```
##
## Call:
## lm(formula = tempdub ~ month. - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.2750 -2.2479  0.1125  1.8896  9.8250
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## month.January    16.608     0.987   16.83  <2e-16 ***
## month.February    20.650     0.987   20.92  <2e-16 ***
## month.March       32.475     0.987   32.90  <2e-16 ***
## month.April       46.525     0.987   47.14  <2e-16 ***
## month.May         58.092     0.987   58.86  <2e-16 ***
## month.June        67.500     0.987   68.39  <2e-16 ***
## month.July        71.717     0.987   72.66  <2e-16 ***
## month.August      69.333     0.987   70.25  <2e-16 ***
## month.September   61.025     0.987   61.83  <2e-16 ***
## month.October     50.975     0.987   51.65  <2e-16 ***
## month.November    36.650     0.987   37.13  <2e-16 ***
## month.December    23.642     0.987   23.95  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.419 on 132 degrees of freedom
## Multiple R-squared:  0.9957, Adjusted R-squared:  0.9953
## F-statistic: 2569 on 12 and 132 DF,  p-value: < 2.2e-16
```

```
fitted(model2)
```

##	1	2	3	4	5	6	7	8
##	16.60833	20.65000	32.47500	46.52500	58.09167	67.50000	71.71667	69.33333
##	9	10	11	12	13	14	15	16
##	61.02500	50.97500	36.65000	23.64167	16.60833	20.65000	32.47500	46.52500
##	17	18	19	20	21	22	23	24
##	58.09167	67.50000	71.71667	69.33333	61.02500	50.97500	36.65000	23.64167
##	25	26	27	28	29	30	31	32
##	16.60833	20.65000	32.47500	46.52500	58.09167	67.50000	71.71667	69.33333
##	33	34	35	36	37	38	39	40
##	61.02500	50.97500	36.65000	23.64167	16.60833	20.65000	32.47500	46.52500
##	41	42	43	44	45	46	47	48
##	58.09167	67.50000	71.71667	69.33333	61.02500	50.97500	36.65000	23.64167
##	49	50	51	52	53	54	55	56
##	16.60833	20.65000	32.47500	46.52500	58.09167	67.50000	71.71667	69.33333
##	57	58	59	60	61	62	63	64
##	61.02500	50.97500	36.65000	23.64167	16.60833	20.65000	32.47500	46.52500
##	65	66	67	68	69	70	71	72
##	58.09167	67.50000	71.71667	69.33333	61.02500	50.97500	36.65000	23.64167
##	73	74	75	76	77	78	79	80
##	16.60833	20.65000	32.47500	46.52500	58.09167	67.50000	71.71667	69.33333
##	81	82	83	84	85	86	87	88
##	61.02500	50.97500	36.65000	23.64167	16.60833	20.65000	32.47500	46.52500
##	89	90	91	92	93	94	95	96
##	58.09167	67.50000	71.71667	69.33333	61.02500	50.97500	36.65000	23.64167
##	97	98	99	100	101	102	103	104
##	16.60833	20.65000	32.47500	46.52500	58.09167	67.50000	71.71667	69.33333
##	105	106	107	108	109	110	111	112
##	61.02500	50.97500	36.65000	23.64167	16.60833	20.65000	32.47500	46.52500
##	113	114	115	116	117	118	119	120
##	58.09167	67.50000	71.71667	69.33333	61.02500	50.97500	36.65000	23.64167
##	121	122	123	124	125	126	127	128
##	16.60833	20.65000	32.47500	46.52500	58.09167	67.50000	71.71667	69.33333
##	129	130	131	132	133	134	135	136
##	61.02500	50.97500	36.65000	23.64167	16.60833	20.65000	32.47500	46.52500
##	137	138	139	140	141	142	143	144
##	58.09167	67.50000	71.71667	69.33333	61.02500	50.97500	36.65000	23.64167

```
residuals(model2)
```

##	1	2	3	4	5	6	7
##	8.0916667	5.0500000	-1.8750000	0.9750000	4.8083333	1.0000000	1.9833333
##	8	9	10	11	12	13	14
##	-1.4333333	0.0750000	-2.4750000	2.9500000	-3.6416667	-0.5083333	-1.5500000
##	15	16	17	18	19	20	21
##	-8.2750000	-1.1250000	3.2083333	-1.0000000	0.3833333	-0.9333333	-0.8250000
##	22	23	24	25	26	27	28
##	-0.0750000	0.7500000	7.4583333	-6.2083333	0.9500000	4.9250000	-1.8250000
##	29	30	31	32	33	34	35
##	-4.8916667	0.5000000	1.9833333	-1.1333333	-0.3250000	-0.7750000	0.5500000
##	36	37	38	39	40	41	42
##	0.9583333	4.8916667	-5.9500000	2.5250000	1.7750000	-4.0916667	0.7000000

```
##          43          44          45          46          47          48          49
## -2.1166667 -3.6333333 -0.2250000 -1.8750000 -3.4500000  2.3583333  2.4916667
##          50          51          52          53          54          55          56
## -0.0500000  7.7250000  3.4750000 -2.7916667  0.2000000 -1.0166667  0.9666667
##          57          58          59          60          61          62          63
## -0.4250000 -0.2750000 -0.8500000 -2.9416667 -2.6083333  3.4500000 -3.0750000
##          64          65          66          67          68          69          70
##  0.0750000  0.5083333 -5.3000000  0.3833333  2.3666667  0.8750000 -3.3750000
##          71          72          73          74          75          76          77
## -2.4500000 -3.2416667 -8.2083333 -1.6500000 -1.0750000  2.1750000  3.5083333
##          78          79          80          81          82          83          84
##  0.6000000  0.4833333  1.2666667  1.4750000  1.7250000  0.0500000  0.1583333
##          85          86          87          88          89          90          91
## -5.4083333 -0.6500000 -2.8750000  1.1750000 -2.2916667  5.7000000 -3.7166667
##          92          93          94          95          96          97          98
## -2.2333333  3.8750000  6.1250000  0.9500000  4.0583333 -3.2083333 -3.4500000
##          99         100         101         102         103         104         105
## -1.6750000 -2.8250000  4.2083333 -1.1000000 -1.5166667  2.2666667  1.0750000
##         106         107         108         109         110         111         112
## -4.9750000 -3.9500000 -6.3416667  5.8916667  5.0500000  9.8250000 -1.3250000
##         113         114         115         116         117         118         119
## -2.5916667  1.4000000  0.5833333  2.9666667  1.4750000  4.6250000  1.3500000
##         120         121         122         123         124         125         126
## -3.2416667  0.9916667 -0.1500000  1.7250000  2.6750000 -3.2916667 -3.7000000
##         127         128         129         130         131         132         133
##  2.2833333 -2.2333333 -3.3250000 -0.1750000  0.1500000  1.8583333  3.7916667
##         134         135         136         137         138         139         140
## -1.0500000 -7.8750000 -5.2250000  3.7083333  1.0000000  0.2833333  1.7666667
##         141         142         143         144
## -3.7250000  1.5250000  3.9500000  2.5583333
```

Exhibit 3.4 on page 33.

```
model3=lm(tempdub~month.) # intercept is automatically included so one month (January) is dropped
summary(model3)
```

```
##
## Call:
## lm(formula = tempdub ~ month.)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.2750 -2.2479  0.1125  1.8896  9.8250
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    16.608     0.987   16.828 < 2e-16 ***
## month.February     4.042     1.396    2.896  0.00443 **
## month.March       15.867     1.396   11.368 < 2e-16 ***
## month.April       29.917     1.396   21.434 < 2e-16 ***
## month.May         41.483     1.396   29.721 < 2e-16 ***
## month.June        50.892     1.396   36.461 < 2e-16 ***
```

```
## month.July      55.108      1.396  39.482 < 2e-16 ***
## month.August    52.725      1.396  37.775 < 2e-16 ***
## month.September 44.417      1.396  31.822 < 2e-16 ***
## month.October   34.367      1.396  24.622 < 2e-16 ***
## month.November  20.042      1.396  14.359 < 2e-16 ***
## month.December   7.033      1.396   5.039 1.51e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.419 on 132 degrees of freedom
## Multiple R-squared:  0.9712, Adjusted R-squared:  0.9688
## F-statistic: 405.1 on 11 and 132 DF,  p-value: < 2.2e-16
```

Exhibit 3.5 on page 35.

```
har.=harmonic(tempdub,1)
model4=lm(tempdub~har.)
summary(model4)
```

```
##
## Call:
## lm(formula = tempdub ~ har.)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.1580  -2.2756  -0.1457   2.3754  11.2671
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    46.2660     0.3088 149.816 < 2e-16 ***
## har.cos(2*pi*t) -26.7079     0.4367 -61.154 < 2e-16 ***
## har.sin(2*pi*t)  -2.1697     0.4367  -4.968 1.93e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.706 on 141 degrees of freedom
## Multiple R-squared:  0.9639, Adjusted R-squared:  0.9634
## F-statistic: 1882 on 2 and 141 DF,  p-value: < 2.2e-16
```

```
M=matrix(1:6,ncol=2)
M
```

```
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

```
dim(M)
```

```
## [1] 3 2
```

```
apply(M,2,mean)
```

```
## [1] 2 5
```

Exhibit 3.6 on page 35.

```
plot(ts(fitted(model4),freq=12,start=c(1964,1)),  
ylab='Temperature',type='l',  
ylim=range(c(fitted(model4),tempdub)))  
points(tempdub)
```

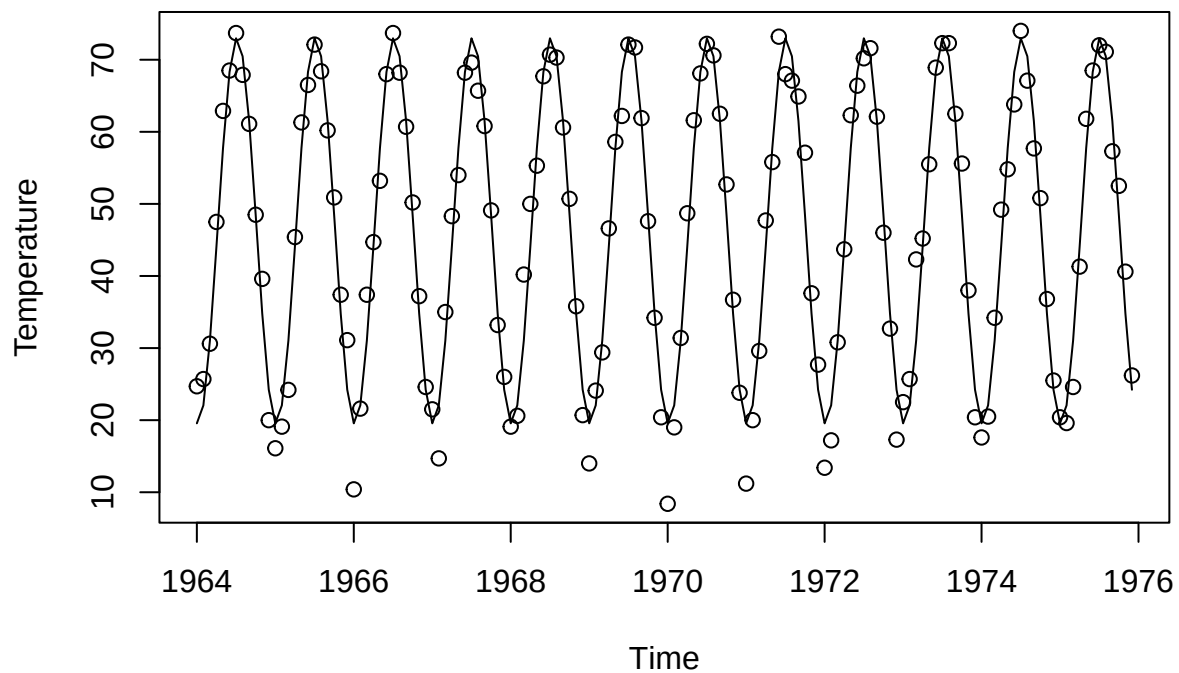


Exhibit 3.8 on page 43.

```
plot(y=rstudent(model3),x=as.vector(time(tempdub)), xlab='Time', ylab='Standardized Residuals',type='o')
```

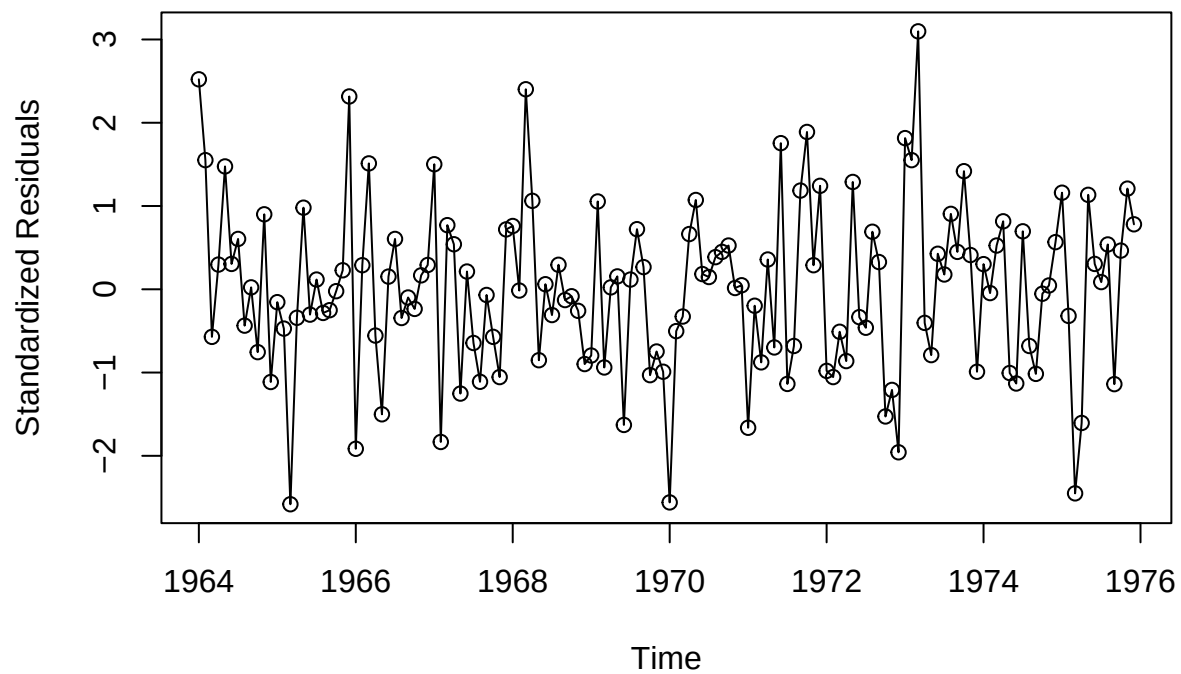


Exhibit 3.11 on page 45.

```
hist(rstudent(model3),xlab='Standardized Residuals')
```

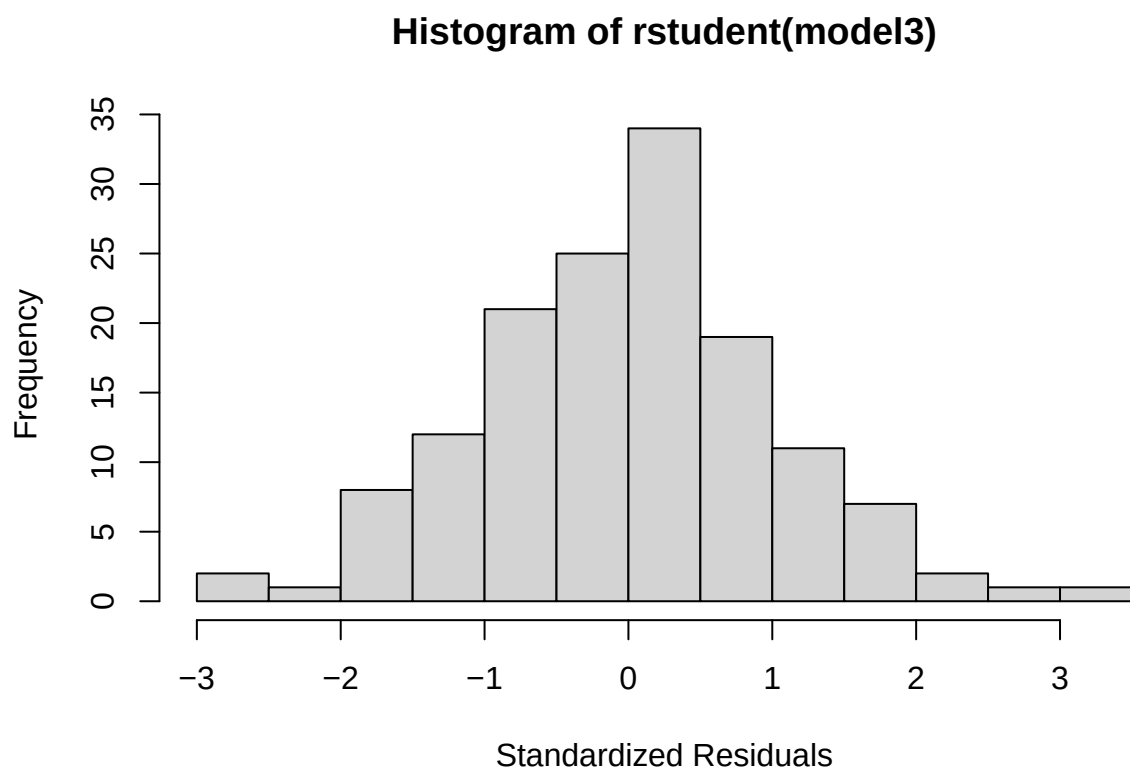


Exhibit 3.12 on page 45.

```
qqnorm(rstudent(model3))
```

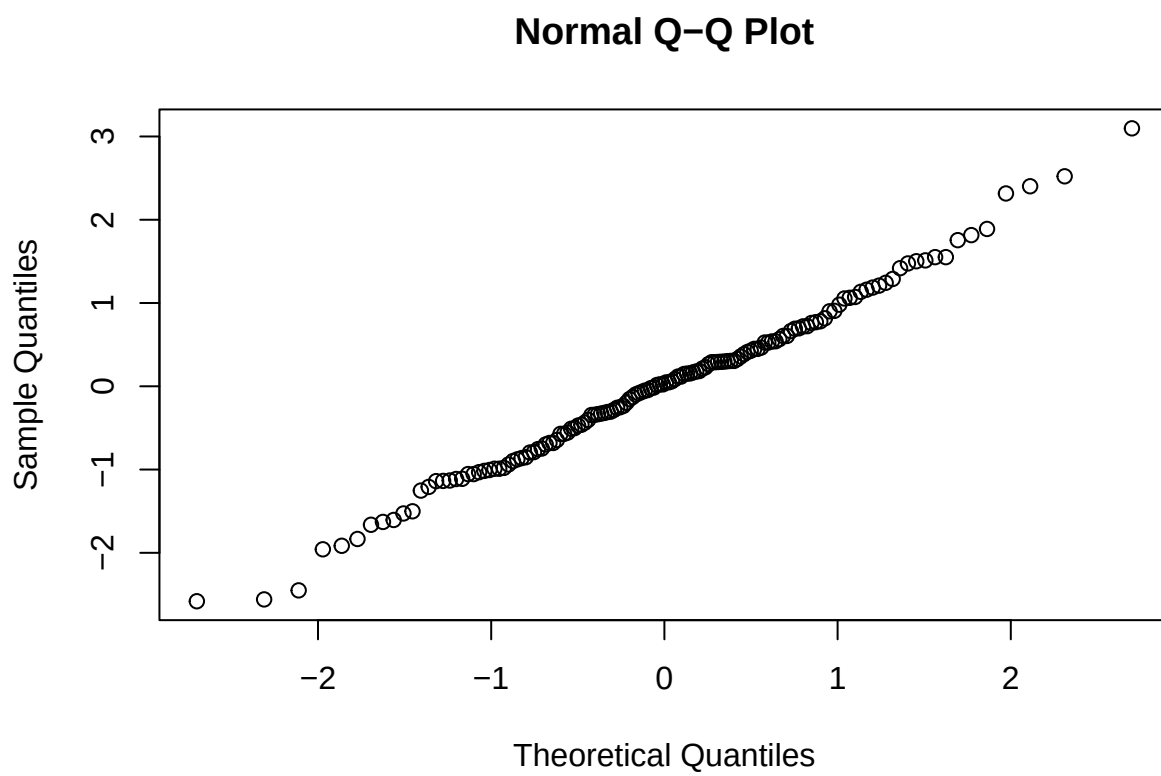
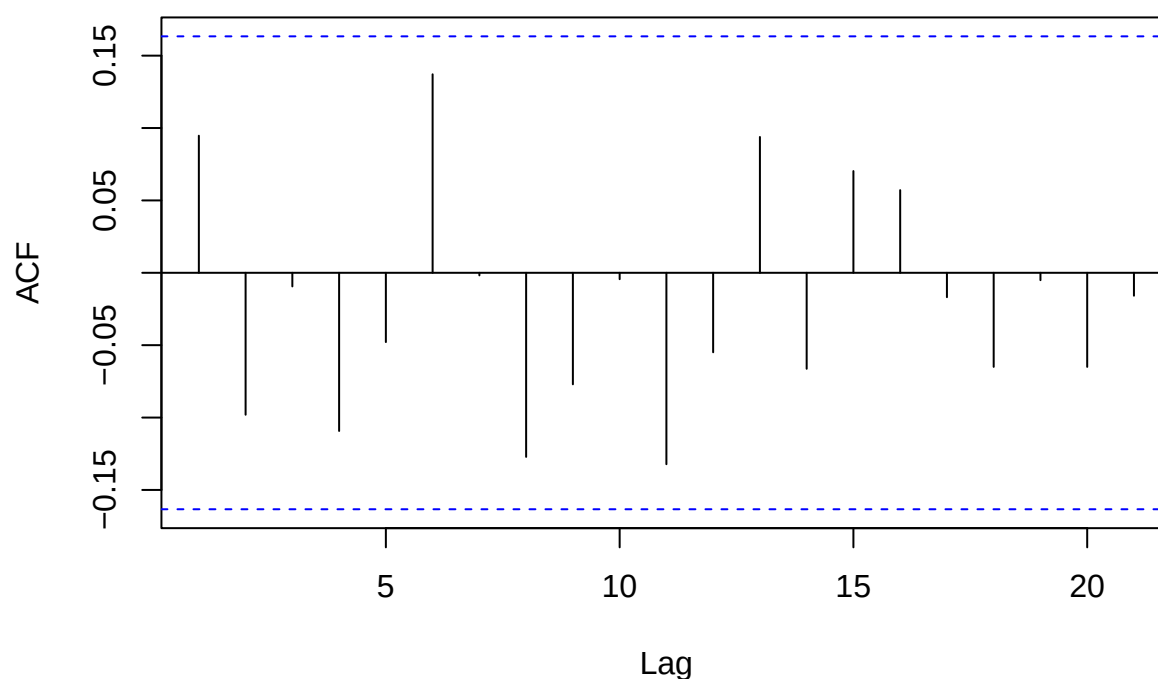


Exhibit 3.13 on page 47.

```
acf(rstudent(model3))
```

Series rstudent(model3)



```
shapiro.test(rstudent(model3))
```

```
##  
##  Shapiro-Wilk normality test  
##  
## data:  rstudent(model3)  
## W = 0.9929, p-value = 0.6954
```

```
runs(rstudent(model3))
```

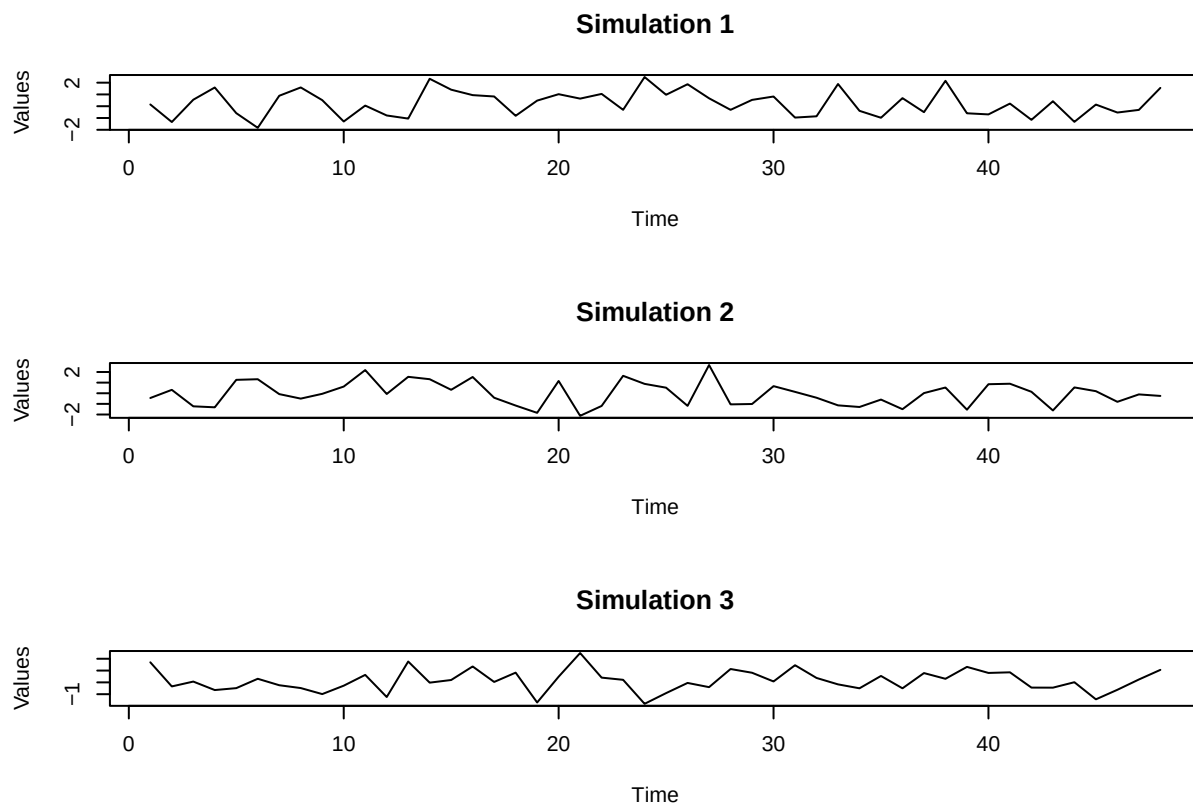
```
## $pvalue  
## [1] 0.216  
##  
## $observed.runs  
## [1] 65  
##  
## $expected.runs  
## [1] 72.875  
##  
## $n1  
## [1] 69  
##  
## $n2  
## [1] 75  
##
```

```
## $k  
## [1] 0
```

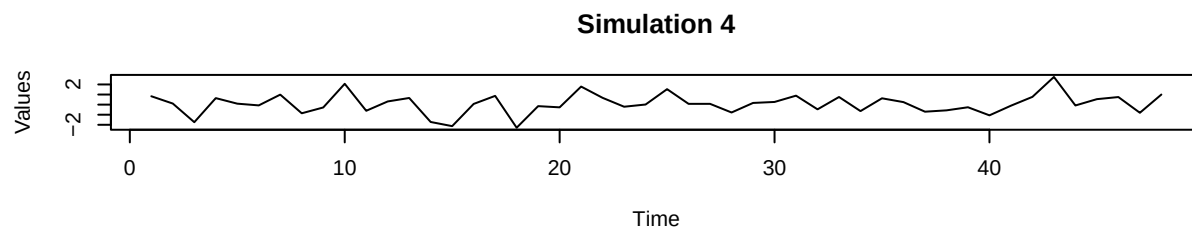
Exercises 1.3-1.5

1.3 Simulate a completely random process of length 48 with independent, normal values. Plot the time series plot. Does it look “random”? Repeat this exercise several times with a new simulation each time.

```
par(mfrow = c(3, 1)) # Plot 3 graphs in 1 column  
  
for (i in 1:4) {  
  random_process <- rnorm(48)  
  
  plot(random_process, type = "l", main = paste("Simulation", i), ylab = "Values", xlab = "Time")  
}
```



```
par(mfrow = c(1, 1))
```



As we can clearly see, the process looks completely random, with the data oscillating evenly around zero and lacking any clear pattern.

```
plot(1:48, type = "n", main = "Multiple Random Simulations", ylab = "Values", xlab = "Time", ylim = c(-2, 2))

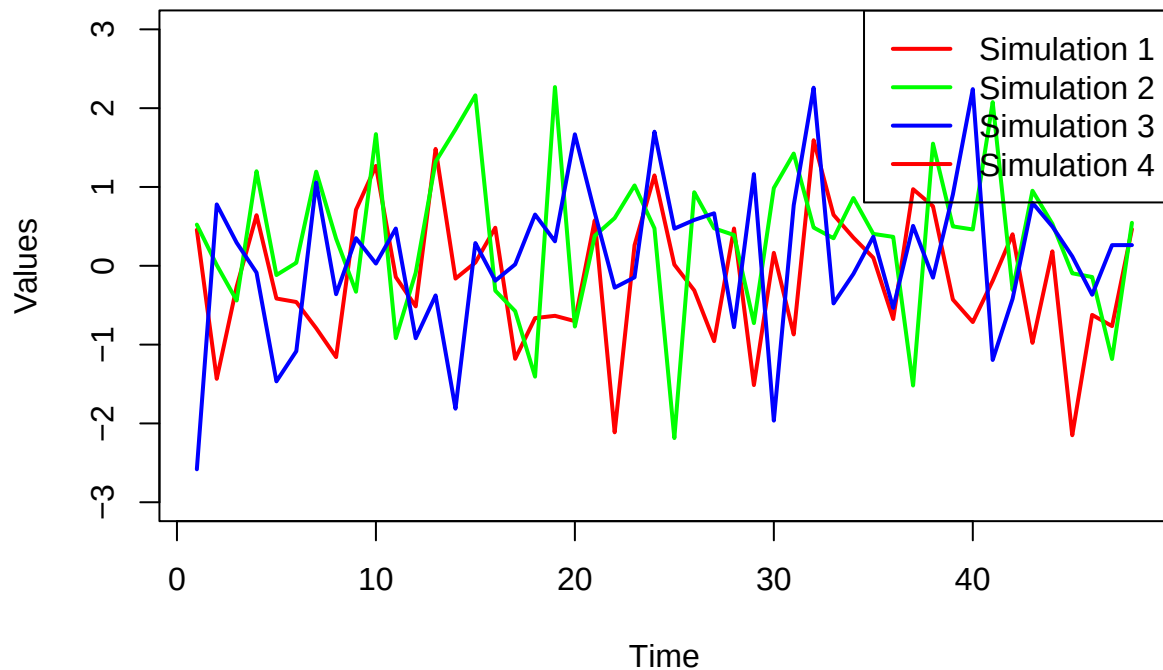
colors <- rainbow(3)

for (i in 1:4) {
  random_process <- rnorm(48)

  lines(random_process, col = colors[i], lwd = 2)
}

legend("topright", legend = paste("Simulation", 1:4), col = colors, lwd = 2)
```

Multiple Random Simulations



1.4. Simulate a completely random process of length 48 with independent, chisquare distributed values, each with 2 degrees of freedom. Display the time series plot. Does it look “random” and non-normal? Repeat this exercise several times with a new simulation each time.

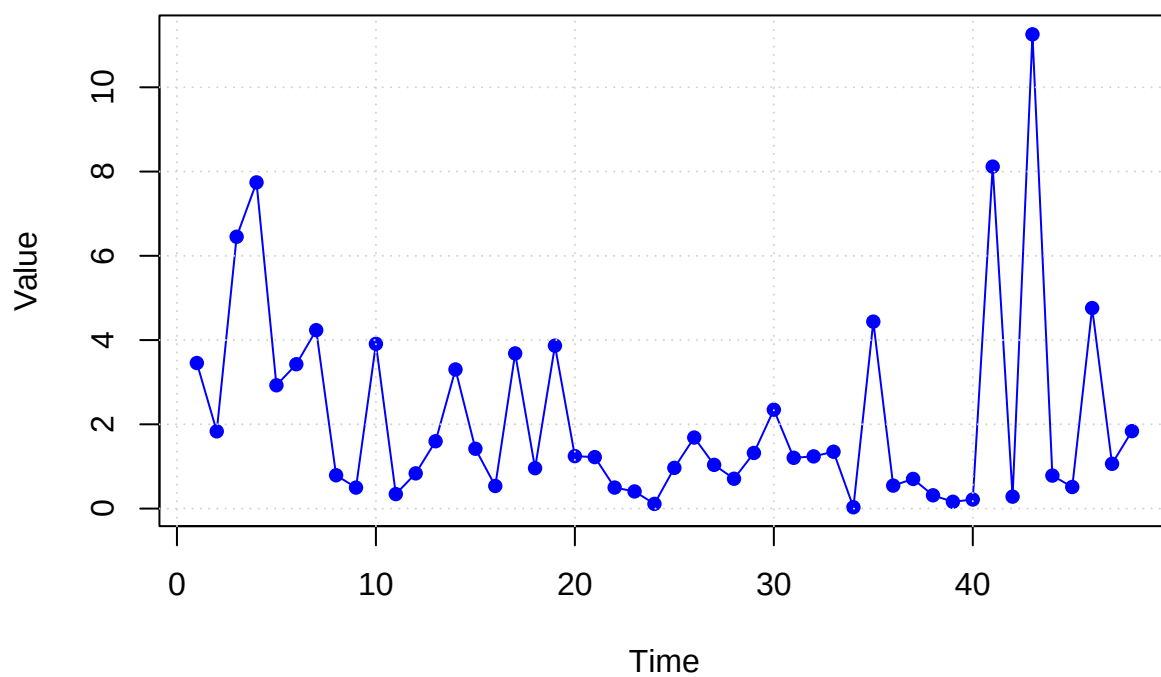
```
length <- 48
df <- 2

simulate_chisquare_process <- function(length, df) {
  rchisq(length, df)
}

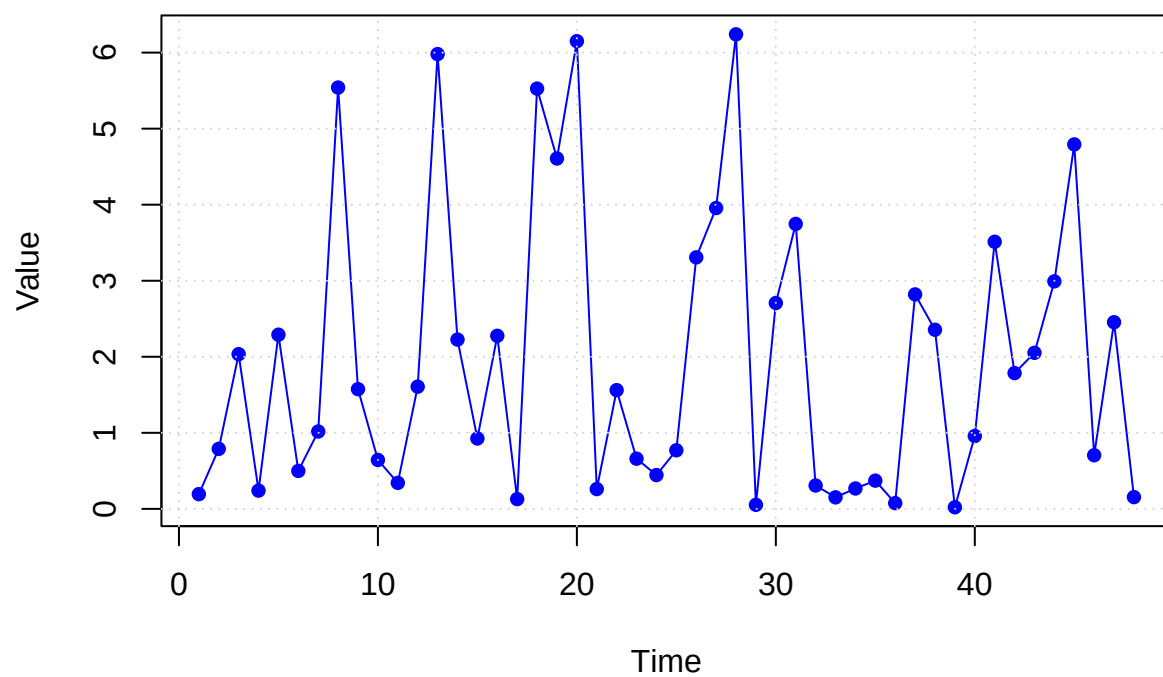
plot_time_series <- function(data, title) {
  plot(data, type = "o", col = "blue", main = title,
       xlab = "Time", ylab = "Value", pch = 16)
  grid()
}

for (i in 1:4) {
  time_series_data <- simulate_chisquare_process(length, df)
  plot_time_series(time_series_data, paste("Chi-Square Distributed Process (2 DoF) - Simulation", i))
}
```

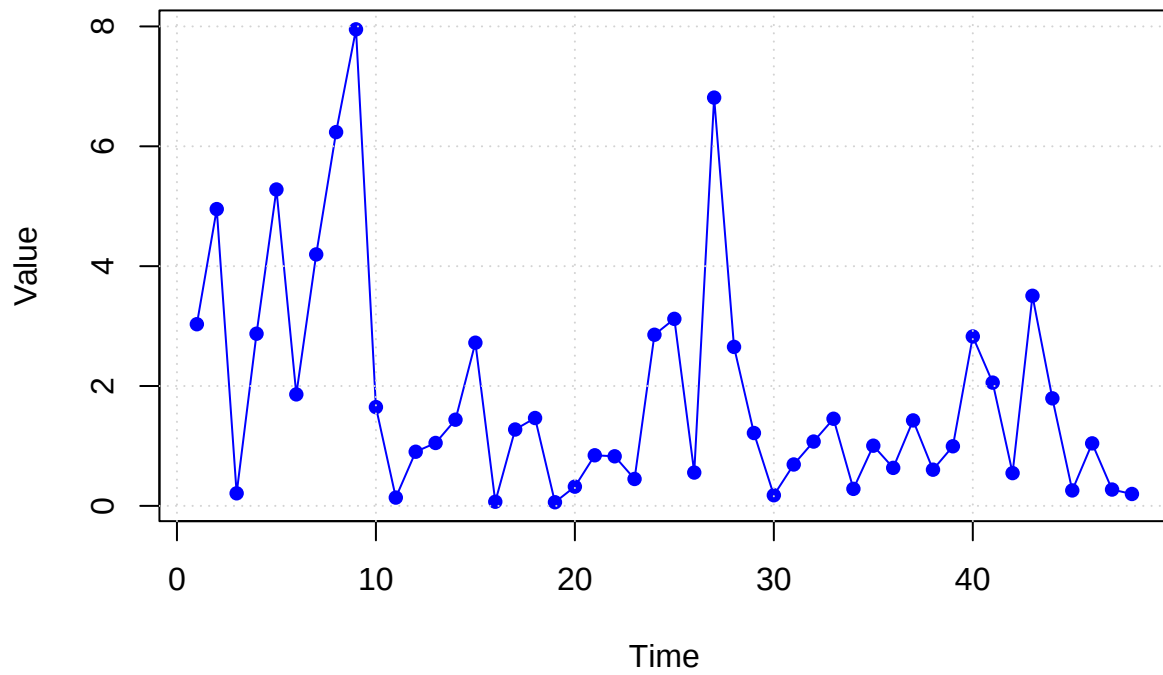
Chi-Square Distributed Process (2 DoF) – Simulation 1



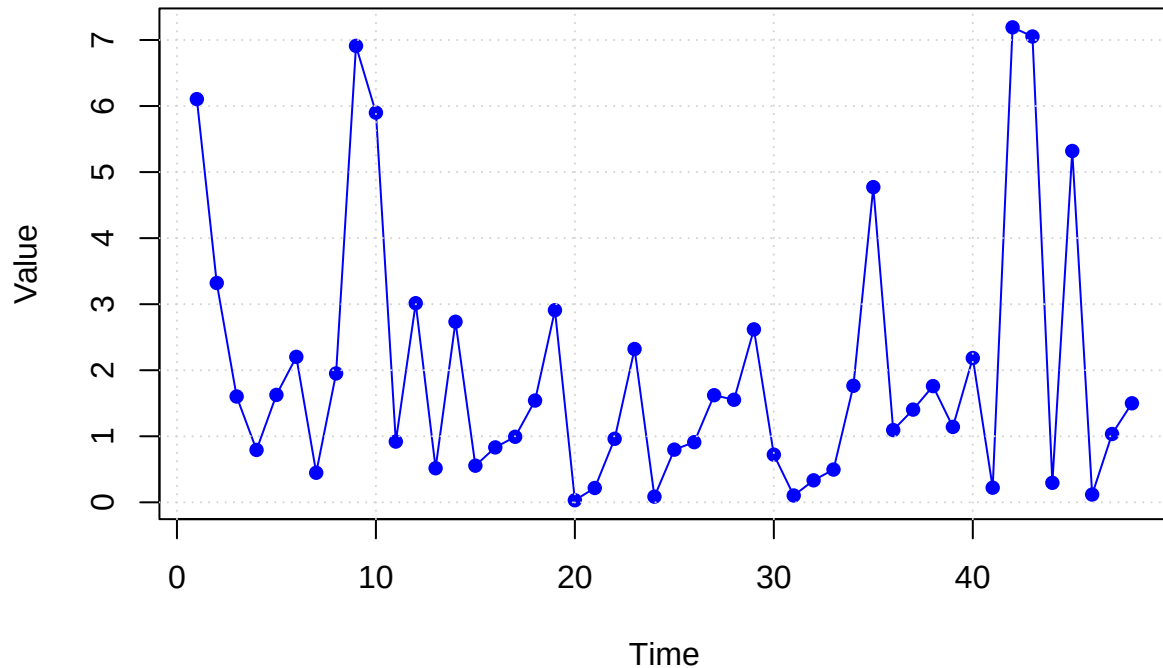
Chi-Square Distributed Process (2 DoF) – Simulation 2



Chi-Square Distributed Process (2 DoF) – Simulation 3



Chi-Square Distributed Process (2 DoF) – Simulation 4



As we can clearly see, the process looks completely random and non-normal, the plot displays only positive values, frequently biased towards the upper end, suggesting a non-normal distribution. While there is an element of randomness, its appearance is notably affected by the skew.

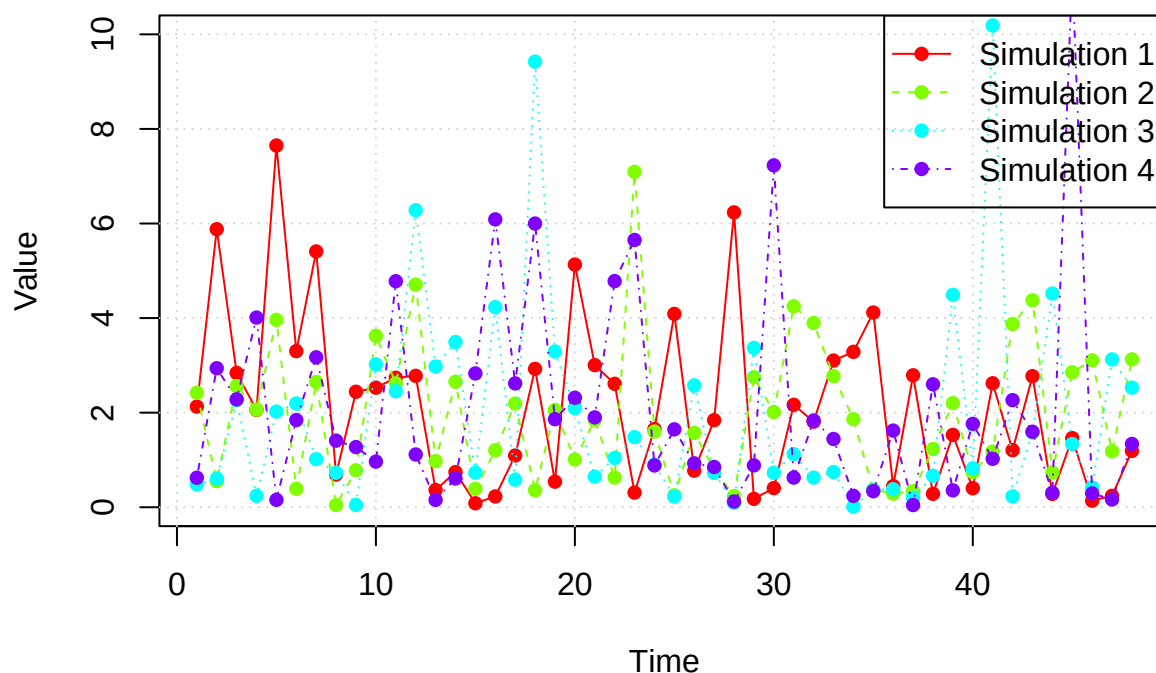
```
plot(1:length, type = "n", ylim = c(0, 10), xlab = "Time",
     ylab = "Value", main = "Combined Chi-Square Distributed Processes (2 DoF)")
grid()

colors <- rainbow(4)

for (i in 1:4) {
  time_series_data <- simulate_chisquare_process(length, df)
  lines(1:length, time_series_data, type = "o", col = colors[i], pch = 16, lty = i)
}

legend("topright", legend = paste("Simulation", 1:4), col = colors, lty = 1:4, pch = 16)
```

Combined Chi-Square Distributed Processes (2 DoF)

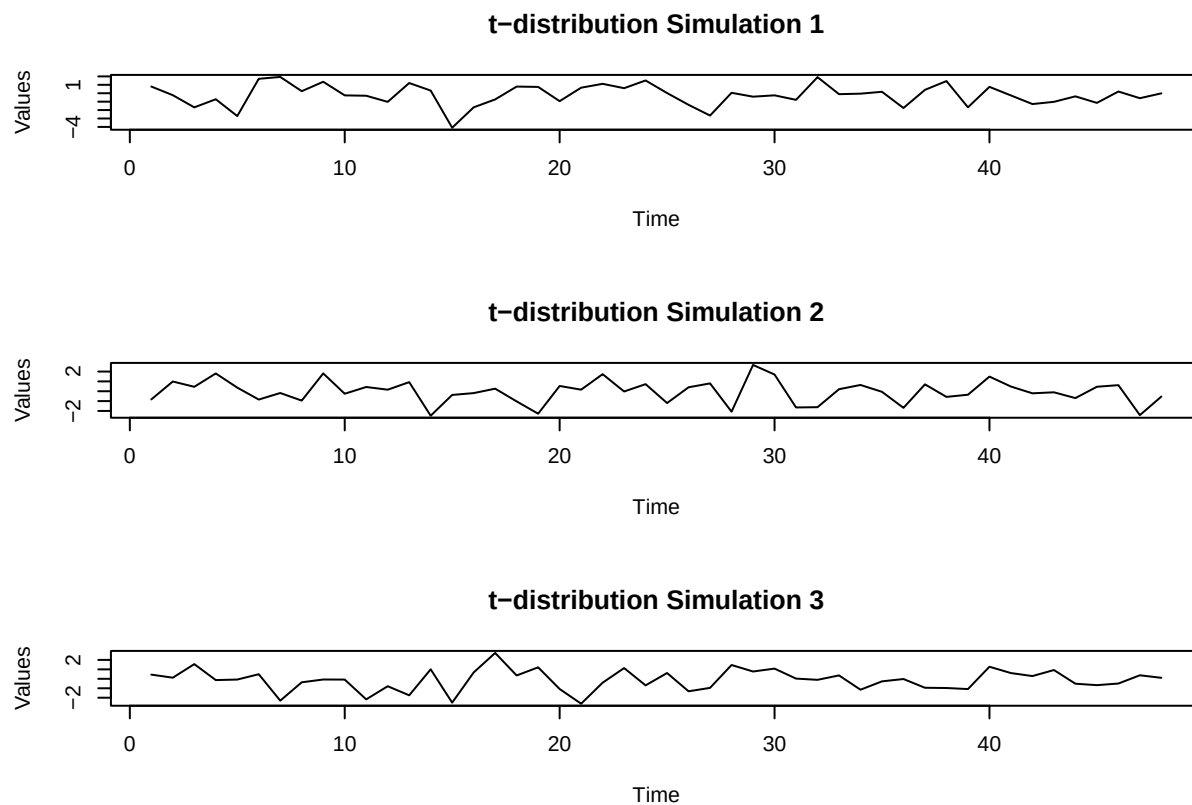


1.5. Simulate a completely random process of length 48 with independent, t-distributed values each with 5 degrees of freedom. Construct the time series plot. Does it look “random” and non-normal? Repeat this exercise several times with a new simulation each time.

```
par(mfrow = c(3, 1))

for (i in 1:3) {
  random_process <- rt(48, df = 5)

  plot(random_process, type = "l", main = paste("t-distribution Simulation", i), ylab = "Values", xlab = "Time")
}
```



```
par(mfrow = c(1, 1))
```

As it is evident the values are totally random and non- normal, the plot resembles the normal distribution but exhibits more pronounced fluctuations because of the heavier tails of the t-distribution. It appears random, albeit with larger occasional spikes

```
colors <- rainbow(3)

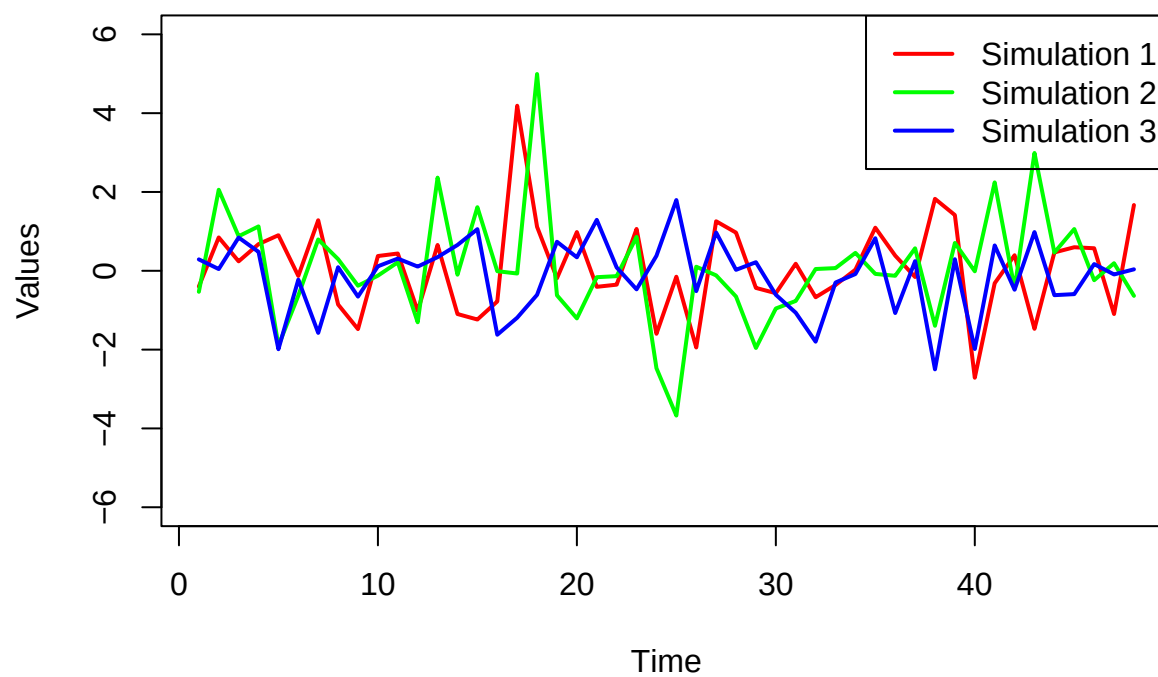
plot(1:48, type = "n", main = "Combined t-distribution Simulations", ylab = "Values", xlab = "Time", ylm = "n")

for (i in 1:3) {
  random_process <- rt(48, df = 5)

  lines(random_process, col = colors[i], lwd = 2)
}

legend("topright", legend = paste("Simulation", 1:3), col = colors, lwd = 2)
```

Combined t-distribution Simulations



```
par(mfrow = c(1, 1))
```